# Technical Proposal: A Stateful 3D Shape Completion Network for Fruit Growth Monitoring

**Date:** November 16, 2024
**Authors:** Based on a collaborative discussion
**Version:** 1.0

## Executive Summary

This document outlines a novel deep learning architecture for accurately monitoring the 3D growth of fruits in a greenhouse environment. Building upon the success of the CoRe++ model for static 3D shape completion, this proposal details a stateful, time-aware network that leverages temporal information to significantly improve measurement accuracy and robustness, especially under conditions of partial occlusion.

The core innovation is a **Time-Aware Fusion Module** that intelligently combines a fruit's past estimated state with its current, partially observed state. To achieve this, the model introduces two key concepts:

- **Disentangled Latent Representation:** The model learns to separate a fruit's intrinsic shape from its variable size, allowing it to enforce the strong prior that shape remains largely consistent while size monotonically increases.
- **Time-Aware Growth Prediction:** The model explicitly incorporates the time delta ($\Delta$t) between observations, enabling it to handle irregular monitoring schedules and learn a more realistic growth model.

This approach transforms the shape completion task from a series of independent predictions into a continuous, dynamic estimation process, resulting in a system that is more accurate, robust to noisy data, and better aligned with the biological reality of fruit growth. This document details the proposed architecture, training strategy, and potential extensions for incorporating a richer history of observations.

## 1. Foundational Concept & Goal

The starting point is a successful implementation of a CoRe-style encoder-decoder network that can complete the 3D shape of a fruit from a single, partial RGB-D image.

- **Current State:** The model is static, treating each observation independently.
- **Limitation:** It does not leverage the powerful information that the same fruit is being observed over time. A heavily occluded fruit at time `t` could be more accurately measured by considering its well-observed shape at `t-1`.

- **Goal:** To evolve the static model into a dynamic, stateful model that maintains a "memory" of each fruit's shape and size, using this history as a prior to refine current measurements.

## 2. Proposed Architecture: The Disentangled, Time-Aware Model

We propose a new architecture that explicitly models the distinct properties of shape and size and is aware of the time elapsed between measurements.

### 2.1. Core Idea 1: Disentangling Shape and Size

The model's latent vector `z` will be split into two components:

- **`z_shape`:** A vector representing the scale-invariant, intrinsic shape of the fruit.
- **`z_size`:** A smaller vector (potentially a single scalar) representing the fruit's current scale or volume.

**Latent Vector Structure:** The final latent representation `z_final` is the **concatenation** of these two components: `z_final = [z_shape, z_size]`. For example, if the total latent dimension is 32: - **Recommended split**: `z_shape` (31-dim) + `z_size` (1-dim), since size can be effectively represented as a scalar - **Alternative**: `z_shape` (28-dim) + `z_size` (4-dim) for richer size representation - The split ratio is a design choice based on the complexity of your fruit shapes

This is achieved by modifying the training process with specialized loss functions:

- **Encoder Architecture:** The CNN encoder will have two output heads, one for `z_shape_obs` and one for `z_size_obs`.
- **Shape Consistency Loss (`L_shape_consistency`):** A loss term that encourages the `z_shape` vectors for the same fruit at different times (`t1`, `t2`) to be as similar as possible. This forces the encoder to learn a size-invariant shape representation. Computed by extracting only the shape portion (e.g., first 31 dims) and calculating: `||z_shape(t1) - z_shape(t2)||²`.
- **Monotonic Size Loss (`L_mono`):** A loss term that penalizes the model if the predicted `z_size` for a fruit decreases over time. Computed by extracting only the size portion (e.g., last dim) and applying a hinge loss: `max(0, z_size(t1) - z_size(t2) + margin)` where `t2 > t1`.

### 2.2. Core Idea 2: The Time-Aware Fusion Module

This new module is responsible for merging the past state with the current observation.

**Inputs:**

- Current observation from the encoder: `z_shape_obs(t)`, `z_size_obs(t)`.
- Refined final state from the previous time step: `z_shape_final(t-1)`, `z_size_final(t-1)`.
- Time difference between observations: `Δt`.

**Architecture:** The module has two specialized heads.

**Shape Fusion Head:**

- **Assumption:** Shape is stable and changes little. The prior `z_shape_final(t-1)` is highly reliable.
- **Mechanism:** Uses a gated mechanism (inspired by GRUs) to compute the new shape as a weighted average of the prior and the new observation: `z_shape_final(t) = α * z_shape_obs(t) + (1 - α) * z_shape_final(t-1)`
  The network learns the gate `α`, which will likely be small, indicating a high trust in the historical shape.

**Size Prediction Head:**

- **Assumption:** Size increases as a function of its previous state and the time elapsed.
- **Mechanism:** An MLP takes `[z_size_obs(t), z_size_final(t-1), Δt]` as input to predict the new `z_size_final(t)`. The time delta `Δt` is **concatenated as a scalar feature** alongside the latent vectors. For example, if `z_size` is 1-dimensional, the MLP input is 3-dimensional: `[current_size, previous_size, time_delta]`.
- **Why Δt matters:** Even with daily observations (~1 day apart), $\Delta$t captures natural irregularities (0.8, 1.2, 2.0 days) and teaches the network realistic growth rates. The model learns that if $\Delta$t=2, it should predict roughly twice the growth compared to $\Delta$t=1, enabling robust handling of irregular monitoring schedules.

### 2.3. Training Workflow (with Ground Truth only for image(t))

The system can be trained end-to-end even with ground truth 3D shapes available only for the current time step `t`.

1. **Prepare a Training Batch:** Each sample consists of `(image(t-1), image(t), Δt, GT_shape(t))`.
2. **Generate Prior State:** Pass `image(t-1)` through the CNN encoder to get `z_obs(t-1)`. These vectors are detached from the computation graph (i.e., gradients will not flow back through them).
3. **Process Current State:** Pass `image(t)` through the CNN encoder to get `z_obs(t)`.
4. **Fuse and Decode:** The Fusion Module uses `z_obs(t)`, the detached `z_obs(t-1)`, and `Δt` to produce the final latent vector `z_final(t)`. This

is fed to the DeepSDF decoder to reconstruct `Predicted_shape(t)`.

5. **Calculate Loss:** The total loss is a weighted sum:
   `L_total = L_reconstruction(Predicted_shape(t), GT_shape(t)) + λ1 * L_shape_consistency + λ2 * L_mono`

   **Loss Computation Details:**

   - **L_reconstruction:** Uses the full concatenated `z_final(t)` → decoder → predicted shape → compare with GT using SDF loss (L1/L2). Gradients flow back through both shape and size components.
   - **L_shape_consistency:** Extracts only the shape portion by slicing `z[:d_shape]` from both `z(t1)` and `z(t2)`, then computes `||z_shape(t1) - z_shape(t2)||²`. This only penalizes shape differences.
   - **L_mono:** Extracts only the size portion by slicing `z[d_shape:]`, then applies hinge loss to ensure monotonic growth. This only constrains size values.

   No one-hot encoding is needed—PyTorch's autograd naturally handles gradients for each sliced portion.

6. **Backpropagate:** Gradients are computed and update the weights of the Fusion Module and the CNN Encoder (based on its processing of `image(t)`).

## 3. Advanced Extension: Incorporating a Richer History (Multi-Step Memory)

While using the immediately preceding state `(t-1)` is powerful, the model's robustness and accuracy can be further enhanced by considering a longer sequence of past observations `(t-N, ..., t-1)`. A richer history allows the model to better estimate growth trends and smooth over noise from isolated, poor-quality observations. Below are two advanced methods for achieving this.

### 3.1. Method 1: Recurrent Neural Network (RNN) for Sequential State Update

The RNN approach provides a classic and highly effective way to process sequential data. It maintains a compact hidden state that acts as an evolving memory, summarizing the entire history of the fruit's growth.

**Core Concept:** The Fusion Module is replaced with a recurrent cell, such as a Gated Recurrent Unit (GRU) or Long Short-Term Memory (LSTM). These are specifically designed to learn temporal dependencies by deciding what information to keep from the past and what to discard.

**Architecture and Workflow:**

1. At time `t`, the CNN encoder produces the current observation vector, `z_obs(t) = [z_shape_obs(t), z_size_obs(t)]`.
2. The GRU/LSTM cell takes two inputs: the current observation `z_obs(t)` concatenated with `Δt`, and its own hidden state from the previous step, `h(t-1)`. This `h(t-1)` vector is the compressed summary of all information up to and including time `t-1`.
3. The cell performs internal calculations using its learned gates (update, reset, etc.) to seamlessly integrate the new information from `[z_obs(t), Δt]` with the historical context in `h(t-1)`.
4. It outputs a new hidden state, `h(t)`. This updated state vector becomes the final, history-aware latent representation `z_final(t)` that is passed to the DeepSDF decoder.

**Training Workflow for RNN Multi-Step Memory:**

1. **Prepare a Sequence Batch:** Each training sample is a temporal sequence: `([image(t-N), ..., image(t-1), image(t)], [Δt₁, ..., Δtₙ], GT_shape(t))`
2. **Initialize Hidden State:** Start with `h(t-N-1) = 0` (zero vector)
3. **Sequential Forward Pass:**
   - For timestep `k` from `t-N` to `t`:
     - Encode: `z_obs(k) = Encoder(image(k))`
     - RNN Update: `h(k) = GRU([z_obs(k), Δt(k-1→k)], h(k-1))`
   - Final hidden state: `h(t)` becomes `z_final(t)`
4. **Decode and Loss:**
   - Decoder produces shape from `z_final(t)`
   - Compute `L_total` against `GT_shape(t)` (only current timestep has GT)
5. **Backpropagation Through Time (BPTT):**
   - Gradients flow backward through the entire RNN sequence
   - Updates encoder, GRU/LSTM cell, and decoder weights

**Advantages:**

- **Elegant State Management:** It elegantly compresses an arbitrarily long history into a single, fixed-size vector, making the system computationally efficient at inference time.
- **Implicit Temporal Ordering:** The sequential nature of the RNN naturally respects the order of observations, giving more implicit weight to recent events.
- **Proven Robustness:** LSTMs and GRUs are the standard for time-series modeling and are robust to challenges like vanishing gradients that plague simple RNNs.
- **Natural Temporal Integration:** The model naturally accumulates information from the entire history through its hidden state.

### 3.2. Method 2: Transformer-based Attention for Dynamic History Focus

The Transformer architecture, particularly its self-attention mechanism, offers a more modern and flexible alternative. Instead of compressing the history into a single state, attention allows the model to dynamically look back at all past data points and decide which ones are most relevant for the current prediction.

**Core Concept:** The model learns to assign "attention scores" to each past observation in the history. This allows it to, for example, ignore a heavily occluded observation at `t-1` and instead focus on a high-quality, clear observation from `t-3` when making its current prediction.

**Architecture and Workflow:**

1. **Inputs:** The module takes the current observation `z_obs(t)` and a list of historical observations `[z_obs(t-N), ..., z_obs(t-1)]`, along with corresponding time deltas `[Δt₁, ..., Δtₙ]`.
2. **Temporal Encoding:** Each observation can be augmented with its time delta: `[z_obs(k), Δt(k-1→k)]` before being processed by the attention mechanism.
3. **Attention Calculation:** `z_obs(t)` serves as the **Query**. The historical observations serve as the **Keys** and **Values**. The model calculates the similarity (e.g., via dot product) between the Query and each Key to produce attention scores.
4. **Context Vector:** These scores are normalized (via softmax) to create weights. A weighted average of the Values is computed, resulting in a single `context_vector`. This vector is a rich summary of the past, emphasizing the most relevant historical moments as determined by the current observation.
5. **Final Fusion:** This `context_vector` is then combined with the current `z_obs(t)` (e.g., via concatenation followed by an MLP) to produce the final latent representation `z_final(t)`.

**Advantages:**

- **Non-Sequential Processing:** It breaks the sequential bottleneck of RNNs, allowing it to directly compare the current state with any point in the past.
- **High Interpretability:** The attention weights can be visualized, providing clear insight into which past time steps the model considered important for its decision.
- **Focus on Quality:** It has the potential to learn to prioritize high-quality, unoccluded past observations, making it extremely robust to intermittent noise.

**Implementation Note:** This approach requires maintaining a sliding window of the last N observations for each fruit. The computational cost scales with the length of this window, but for typical growth monitoring, this is highly

manageable.

## 4. Overall System Architecture Diagram

```
graph TD
    subgraph "Time t"
        A1[Image(t)] --> B(CNN Encoder);
        B --> C1[z_shape_obs(t)];
        B --> C2[z_size_obs(t)];
        C1 & C2 --> D{Time-Aware<br>Fusion Module};
    end

    subgraph "Time t-1 (Prior State)"
        A2[Image(t-1)] --> B2(CNN Encoder);
        B2 --> E1[z_shape_final(t-1)];
        B2 --> E2[z_size_final(t-1)];
        E1 & E2 --> D;
    end

    subgraph "Time Information"
        F[Δt] --> D;
    end

    D --> G([z_shape_final(t), z_size_final(t)]);
    G --> H(Concatenate);
    H --> I(DeepSDF Decoder);
    I --> J(Final 3D Shape);

    J -- Loss Calculation --> K((Loss));
    subgraph "Ground Truth"
        L[GT_shape(t)] -- Loss Calculation --> K;
    end

    style B fill:#f9f,stroke:#333,stroke-width:2px
    style B2 fill:#f9f,stroke:#333,stroke-width:2px
    style D fill:#bbf,stroke:#333,stroke-width:2px
    style I fill:#9f9,stroke:#333,stroke-width:2px
```

*Note: For training, the path from A2 to E2 is used to generate a detached prior input.*

## 5. Implementation Details and Design Choices

### 5.1. Latent Vector Representation

The final latent vector is structured as a concatenation: `z_final = [z_shape, z_size]`

**Recommended Dimension Split:** - For a 32-dimensional latent space: `z_shape` (31-dim) + `z_size` (1-dim) - Alternative for complex shapes: `z_shape` (28-dim) + `z_size` (4-dim)

The decoder receives the full concatenated vector, while individual loss functions access their relevant portions via slicing: - Shape operations: `z[:d_shape]` - Size operations: `z[d_shape:]`

### 5.2. Time Delta (Δt) Integration

The time delta between observations is a critical feature that captures growth dynamics:

**Usage Pattern:** - **Two-step model:** Concatenated with size latents as input to MLP: `[z_size_obs(t), z_size_final(t-1), Δt]` - **RNN model:** Concatenated with observation at each timestep: `[z_obs(k), Δt(k-1→k)]` - **Transformer model:** Augments each historical observation: `[z_obs(k), Δt(k-1→k)]`

**Practical Values:** - With daily monitoring: Δt 1.0 day (regular case) - Captures irregularities: Δt {0.8, 1.2, 1.5, 2.0, 3.0} days - Units can be hours, days, or normalized time—consistency matters

**Why it's essential:** The model learns growth as a function of time, enabling it to distinguish between "1 day of growth" vs "3 days of growth" and make proportionally scaled predictions.

### 5.3. Loss Function Implementation

Each loss component operates on specific slices of the latent vector:

**L_reconstruction (Primary Loss):** - Input: Full `z_final(t) = [z_shape, z_size]` - Process: Decoder → Predicted SDF → Compare with GT - Metric: L1 or L2 distance on signed distance values - Gradients: Flow to both shape and size components

**L_shape_consistency (Disentanglement Loss):** - Input: `z_shape(t1)` and `z_shape(t2)` (extracted via `z[:d_shape]`) - Metric: `||z_shape(t1) - z_shape(t2)||²` or cosine similarity - Effect: Forces shape representation to be size-invariant - Weight: `λ1` (typically 0.1-1.0)

**L_mono (Growth Constraint Loss):** - Input: `z_size(t1)` and `z_size(t2)` (extracted via `z[d_shape:]`) - Metric: Hinge loss `max(0, z_size(t1) - z_size(t2) + margin)` for t2 > t1 - Effect: Enforces monotonic growth over time - Weight: `λ2` (typically 0.5-2.0)

**Total Loss:** `L_total = L_reconstruction + λ1 * L_shape_consistency + λ2 * L_mono`

PyTorch's automatic differentiation handles gradient routing through sliced operations without requiring manual indexing or one-hot encodings.

## 6. Conclusion

The proposed stateful architecture represents a significant advancement over static shape completion models for agricultural monitoring. By explicitly disentangling shape and size and incorporating temporal awareness, the system is designed to be more accurate, robust to real-world challenges like occlusion and irregular data capture, and ultimately provide a more reliable tool for yield estimation and growth analysis. This document serves as a blueprint for the development and training of such a system.